```
import sqlite3
from PySide6.QtCore import Qt

# Explicit Architecture Mapping Data Roles
ROLE_UID_DATA = Qt.UserRole  # Dictionary: {"id": int, "path": str, "line": int, "col": int}

class IndexTreePersistence:
    """Handles serialization and top-down reconstruction of the multi-column IDn tree."""

    def save_to_db(model, db_path: str, project_name: str, settings_dict: dict):
        """Flattens hierarchical multi-column headings and reference tokens into an SQLite table."""
        import sqlite3
        from PySide6.QtCore import Qt

        # Explicitly define our data anchor role to pull metadata records cleanly
        ROLE_UID_DATA = Qt.ItemDataRole.UserRole + 1

        conn = sqlite3.connect(db_path)
        try:
            cursor = conn.cursor()

            # 1. Materialize clean metadata schemas
            cursor.execute("DROP TABLE IF EXISTS project_metadata")
            cursor.execute("""
                CREATE TABLE project_metadata (
                    key TEXT PRIMARY KEY,
                    value TEXT
                )""")

            cursor.execute("INSERT INTO project_metadata (key, value) VALUES (?, ?)", ("project_name", project_name))

            for setting_key, setting_value in settings_dict.items():
                cursor.execute("""
                    INSERT INTO project_metadata (key, value)
                    VALUES (?, ?)
                """, (f"cfg_{setting_key}", str(setting_value)))

            # Initialize pristine database table instances
            cursor.execute("DROP TABLE IF EXISTS index_references")
            cursor.execute("DROP TABLE IF EXISTS index_headings")

            cursor.execute("""
                CREATE TABLE index_headings (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    parent_id INTEGER,
                    name TEXT,
                    depth INTEGER
                )""")

            cursor.execute("""
                CREATE TABLE index_references (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    heading_id INTEGER,
                    unique_id_number INTEGER,
                    file_path TEXT,
                    line INTEGER,
                    col INTEGER,
                    absolute_position INTEGER,
                    encap TEXT,
                    FOREIGN KEY(heading_id) REFERENCES index_headings(id) ON DELETE CASCADE
                )""")

            # Recursive depth-first traversal tracking full ancestry path names
            def serialize_row_node(parent_qt_item, parent_db_id=None, current_depth=0, path_segments=None):
                if path_segments is None:
                    path_segments = []

                for i in range(parent_qt_item.rowCount()):
                    # Column 0 manages the structural header term branch
                    heading_item = parent_qt_item.child(i, 0)
                    if not heading_item:
                        continue

                    current_node_text = heading_item.text().strip()

                    # Accumulate name segments to build the absolute unified index entry path
                    node_full_path_list = path_segments + [current_node_text]
                    full_heading_path_str = "!".join(node_full_path_list)

                    # 1. Store the fully qualified structural path string representation
                    cursor.execute("""
                        INSERT INTO index_headings (parent_id, name, depth)
                        VALUES (?, ?, ?)
                    """, (parent_db_id, full_heading_path_str, current_depth))

                    heading_db_id = cursor.lastrowid
```

```
                # 2. Extract merged reference data explicitly from Column 1 on this same row level
                row_parent = heading_item.parent() or model.invisibleRootItem()
                row_index = heading_item.row()

                reference_cell = row_parent.child(row_index, 1)
                if reference_cell:
                    # Extract the array collection list stored in UserRole + 1
                    records_list = reference_cell.data(ROLE_UID_DATA)

                    if records_list and isinstance(records_list, list):
                        for rec in records_list:
                            if not rec:
                                continue

                            # Resolve parameter naming mutations across data layers smoothly
                            uid_num = rec.get("unique_id_number") or rec.get("id") or 0
                            f_path = rec.get("file_path") or rec.get("path") or ""
                            line_coord = rec.get("line_number") or rec.get("line") or 1
                            col_coord = rec.get("column_offset") or rec.get("col") or 0

                            # ------------------------------------------------------------
                            # MODIFICATION 1: EXTRACT NEW ARCHITECTURAL KEYS FROM RECORD
                            # ------------------------------------------------------------
                            abs_pos = rec.get("absolute_position") # Extracts raw character index integer
                            encap_val = rec.get("encap") or "standard" # Guard against breaking None values

                            # Convert absolute position safely to an int or preserve NULL if it's completely missing
                            safe_abs_pos = int(abs_pos) if abs_pos is not None else None

                            # ------------------------------------------------------------
                            # MODIFICATION 2: EXPAND SQL INSERT QUERY TO MAP NEW FIELDS
                            # ------------------------------------------------------------
                            cursor.execute("""
                                INSERT INTO index_references (
                                    heading_id, unique_id_number, file_path, line, col, absolute_position, encap
                                ) VALUES (?, ?, ?, ?, ?, ?, ?)
                            """, (
                                heading_db_id,
                                int(uid_num),
                                str(f_path),
                                int(line_coord),
                                int(col_coord),
                                safe_abs_pos, # Maps correctly to database column index 6
                                str(encap_val) # Maps correctly to database column index 7
                            ))

                # 3. Recurse down deeper if this structural parent contains sub-branches
                if heading_item.rowCount() > 0:
                    serialize_row_node(heading_item, heading_db_id, current_depth + 1, node_full_path_list)

        serialize_row_node(model.invisibleRootItem())
        conn.commit()
    finally:
        conn.close()

@staticmethod
def load_from_db(model, view, db_path: str):
    """Reconstructs multi-column alignment structures, ensuring parents exist before anchoring references."""
    import os
    import sqlite3
    from PySide6.QtCore import Qt
    from PySide6.QtGui import QStandardItem
    from PySide6.QtWidgets import QStyle
    from IndexTreeView import CaseInsensitiveItem  # Adjust module paths accordingly

    # Explicitly define our data anchor role to inject metadata records cleanly
    ROLE_UID_DATA = Qt.ItemDataRole.UserRole + 1

    # Engage execution optimization blocks to bypass visual recalculations during batch loops
    view.setSortingEnabled(False)
    model.clear()
    model.setHorizontalHeaderLabels(["Index Terms", "References"])

    if not os.path.exists(db_path):
        view.setSortingEnabled(True)
        return

    try:
        conn = sqlite3.connect(db_path)
        conn.row_factory = sqlite3.Row
        cursor = conn.cursor()

        # Fetch relational data arrays down stream
        headings = [dict(r) for r in cursor.execute("SELECT * FROM index_headings").fetchall()]
        references = [dict(r) for r in cursor.execute("SELECT * FROM index_references").fetchall()]
        conn.close()
```

```
except sqlite3.OperationalError:
    view.setSortingEnabled(True)
    return

if not headings:
    view.setSortingEnabled(True)
    return

# 1. Group references cleanly by heading ID linkage arrays and normalize schemas
ref_lookup_map = {}
for r in references:
    h_id = r['heading_id']
    line_val = r['line']
    col_val = r['col']
    f_path = r['file_path']
    uid_num = r['unique_id_number']

    # ------------------------------------------------------------
    # MODIFICATION 1: EXTRACT THE PERSISTED SQL ARCHITECTURAL COLUMNS
    # ------------------------------------------------------------
    # Safe parsing ensures that if older DB files don't have absolute_position, it falls back to None safely
    abs_pos_val = r.get('absolute_position')
    encap_val = r.get('encap') or "standard"

    ref_record = {
        "uid": f"{f_path}:{line_val}:{col_val}",
        "unique_id_number": int(uid_num),
        "id": int(uid_num), # Schema mapping alias duplicate matching
        "file_path": str(f_path),
        "line_number": int(line_val),
        "column_offset": int(col_val),
        "line": int(line_val),
        "col": int(col_val),
        # ------------------------------------------------------------
        # MODIFICATION 2: RE-ANCHOR INTO THE ACTIVE DATA MODEL DICTIONARY
        # ------------------------------------------------------------
        "absolute_position": int(abs_pos_val) if abs_pos_val is not None else None,
        "encap": str(encap_val)
    }
    ref_lookup_map.setdefault(h_id, []).append(ref_record)

# 2. Reconstruct parent-child relations based on fully qualified path text keys
# This approach ensures your hierarchy loads cleanly without structural drift
path_registry = {}

# Sort headings by depth to ensure that higher-level parent nodes are built before subheadings
sorted_headings = sorted(headings, key=lambda x: x.get("depth", 0))

for h in sorted_headings:
    h_id = h['id']
    full_path_str = h['name'] or ""
    if not full_path_str:
        continue

    # Split path text into separate tiers (e.g., "Animals!Mammals" -> ["Animals", "Mammals"])
    parts = [p.strip() for p in full_path_str.split("!") if p.strip()]
    if not parts:
        continue

    parent_item = model.invisibleRootItem()
    current_accumulated_path = []

    # 3. Step down the hierarchy, finding or creating nodes layer by layer
    for i, token in enumerate(parts):
        current_accumulated_path.append(token.lower())
        path_key = tuple(current_accumulated_path)

        if path_key not in path_registry:
            # Column 0 manages the heading branch case-insensitively via CaseInsensitiveItem
            branch_item = CaseInsensitiveItem(token)
            branch_item.setEditable(False)
            branch_item.setIcon(view.style().standardIcon(QStyle.SP_DirIcon))

            # Column 1 serves as the structural reference token holder
            ref_item = QStandardItem("")
            ref_item.setEditable(False)

            parent_item.appendRow([branch_item, ref_item])
            path_registry[path_key] = branch_item

        parent_item = path_registry[path_key]

    # 4. Leaf Node Target reached: Retrieve references and inject into Column 1 companion item
    attached_references = ref_lookup_map.get(h_id, [])
    if attached_references:
        row_idx = parent_item.row()
        actual_parent = parent_item.parent() or model.invisibleRootItem()
```

```
            sibling_ref_item = actual_parent.child(row_idx, 1)

            if sibling_ref_item:
                # Save the complete metadata array list inside the user data role context
                sibling_ref_item.setData(attached_references, ROLE_UID_DATA)

                # Sort unique ID numbers and format them cleanly as selectable bracket tokens
                unique_ids = sorted(list(set(r["unique_id_number"] for r in attached_references)))
                token_str = " ".join(f"[{uid}]" for uid in unique_ids)

                sibling_ref_item.setText(token_str)
                sibling_ref_item.setIcon(view.style().standardIcon(QStyle.SP_FileIcon))

    # Disengage optimization blocks, run alphabetical item resolution, and expand view lanes
    view.setSortingEnabled(True)
    model.sort(0, Qt.AscendingOrder)
    view.expandAll()

@staticmethod
def read_metadata(db_path: str) -> tuple[str, dict]:
    """
    Queries the database footprint cleanly to isolate and unpack settings configurations.
    Allows the UI to parse preferences before background thread worker runs.
    """
    extracted_name = ""
    extracted_settings = {}

    conn = sqlite3.connect(db_path)
    try:
        cursor = conn.cursor()
        cursor.execute("SELECT key, value FROM project_metadata")
        rows = cursor.fetchall()

        for key_str, value_str in rows:
            if key_str == "project_name":
                extracted_name = value_str
            elif key_str.startswith("cfg_"):
                # Strip out the configuration key prefix to restore internal register codes
                actual_key = key_str[4:]
                extracted_settings[actual_key] = value_str
    except sqlite3.OperationalError:
        pass # Return empty elements if table layout formats are missing
    finally:
        conn.close()

    return extracted_name, extracted_settings
```